

Backend Capstone

You have walked the whole backend road: Python, FastAPI end to end, SQLAlchemy, PostgreSQL, Firestore, three styles of authentication, caching, indexing, Docker and CI. The capstone is where it becomes engineering. Your team receives a **product brief** written the way a startup would hand it to its backend team. Read the next four pages before you read your own brief — they explain how a brief is laid out, how you will be assessed, and the code standards every team is held to.

Team assignment

#	Team	Members	Product	The hard problem
1	Lion Moons	Gabriel & Loveth	MediStock — Pharmacy Inventory	Exactly once, and first-to-expire first

How to read your brief

Every brief has the same twelve parts, in the same order. If a part confuses you, the table below says what it is for and what to do with it. Read the whole brief twice before you write anything — once to understand the business, once to plan the build.

In one minute	The product in plain words. If you cannot explain your project to a friend using only this paragraph, read it again.
Words you need	A small glossary. Every technical word in the brief is explained here first.
A day in the life	A short story of real people using the product. This is the most important section — it shows exactly what “working” looks like.
The problem	Why the product exists. Use it to decide what matters when you run out of time.
Who uses it	The roles. Every role becomes a value in your <code>users.role</code> column and a rule in your permission checks.
User stories	One sentence per feature: who wants it, what, and why. Each story becomes one or more endpoints. Build them in the order listed.
Acceptance criteria	Given / When / Then. These are literally the tests the examiner will run. Turn every line into a pytest test — that is your test plan, written for you.
How the key request flows	A flowchart of the most important write in your product, step by step, with the status code for every failure. Copy its shape into your service function.
The hard problem	One thing that is genuinely difficult and that real companies pay for. It has its own flowchart and five required tests. Do it in week two, after everything else works.
Data model	Your tables, their columns, relationships, UNIQUE rules and indexes. Draw this on paper on Day 1 before touching <code>SQLModel</code> .
API surface	Every endpoint: method, path, who may call it, success code, error codes. Your <code>/docs</code> page should match this table when you are done.
Backend topics	The list of concepts this project proves you understand. Expect a question on each one in your viva.

How you will be assessed — read carefully

This capstone is graded on **understanding**, not on the amount of code that exists. Code you cannot explain line by line, alone, at a whiteboard, earns nothing — however good it looks. The rules below exist so that the only way to pass is to actually learn. They are not optional.

1. Paper before code	Day 1–2 you produce, by hand: an ERD, a list of endpoints with roles, and the hard problem explained in your own words. Your instructor signs it. No commit is accepted before the signature.
2. Small commits, every day	Commit at least three times a day with messages that say what and why (“add hold expiry sweep; holds older than 10 min are deleted”). One giant commit of finished code is a fail. Your Git history is part of your grade.
3. Daily log	Five lines in LOG.md at the end of every day: what you did, what broke, what you learnt, what is next, who did what. Written by the person who did the work.
4. Tests first for the hard problem	The five required tests for the hard problem must be committed — failing — before the code that makes them pass. The commit history must show it.
5. Weekly viva	End of week 1 and week 2, each member alone: 10 minutes, a random file opened, “explain this function; why is it here; what happens if I delete this line?” No notes, no laptop of your own.
6. Live change at the demo	At the final demo the examiner asks for one small change on the spot — a new field, a changed rule, a new 409 case — and you make it live, with a test. Fifteen minutes.
7. Swap and extend	On Day 7 you swap one module with your partner and add a feature to it. You must be able to read your partner's code as easily as your own — that is what clean code means.
8. Own words	The README sections “The hard problem” and “Why this design” must be in your own words. If it reads like a manual, expect to be asked to rewrite it at the whiteboard.

Clean code & design standards

These are the rules professional teams work by. Each one will be checked in code review. If a rule is unclear, ask on Day 1, not on Day 13.

Layers	routers □ services □ repositories/models. A route function parses the request and calls one service function. All business rules live in services. Database queries live in one place per table. Nothing else touches the session.
Naming	Names say what things are: <code>create_hold()</code> , <code>release_expired_holds()</code> , <code>RoomNight</code> , <code>is_available</code> . No <code>x</code> , <code>tmp</code> , <code>data2</code> , <code>doStuff</code> . Functions are verbs; classes are nouns; booleans read as questions.
Small functions	A function does one thing and fits on one screen (about 30 lines). If you need a comment to separate sections inside it, split it.
Models in and out	Separate request and response models. A response model never contains <code>password_hash</code> or internal fields nobody asked for. Use <code>response_model</code> on every route.
Errors	One error shape for the whole API: <code>{"error": {"code": "...", "message": "...", "request_id": "..."} }</code> . Raise <code>HTTPException</code> from services with the right code; never return error dicts with 200.
Transactions	One transaction per business action. Begin it in the service, commit once, roll back on any error. Never half-save.
Configuration	Everything that differs between your laptop and production comes from environment variables through <code>pydantic-settings</code> . No secret, URL or key in code. <code>.env.example</code> is committed; <code>.env</code> is not.
Dependencies	<code>get_db</code> , <code>get_current_user</code> , <code>require_role(...)</code> are dependencies, declared once, reused everywhere. No copy-pasted auth checks inside route bodies.
Tests	Tests read like the acceptance criteria: <code>test_second_hold_on_same_room_night_returns_409()</code> . Each test sets up its own data. A test database, never the real one.
Docs	Every route has a summary and tags. Request models carry a filled-in example. <code>/docs</code> must be usable by a stranger without asking you anything.
Twelve-factor habits	Stateless app (no data kept in memory between requests), logs to <code>stdout</code> , config in <code>env</code> , one codebase, dev and prod as alike as Docker can make them.
Design principles	Single responsibility everywhere. A service does not know it is called by HTTP. Comment the why only when it is surprising; let names carry the what.

Two standard skills the project includes

Both are new, both are everywhere in real backends, and both are two evenings of reading. Every team implements both in the same standard way so that they can be examined the same way.

The payment webhook

Until now your API has always been the one being called. A **webhook** is the opposite: the payment provider calls *you* when something happens (“payment succeeded”). Three facts change everything: the caller is a machine, so there is no login — it proves itself with a **signature**; it may call **twice** for the same event, so you must ignore repeats; and it expects a fast **200**, so heavy work goes to a background task. The class uses a small **mock provider** script (given to you) that sends signed webhooks, including deliberate duplicates and one with a bad signature. Your endpoint:

Endpoint	POST /api/v1/webhooks/payment — no JWT.
Header	X-Signature: HMAC-SHA256 of the raw request body, using your WEBHOOK_SECRET. Compare with hmac.compare_digest. Wrong □ 401.
Body	{event_id, type: "payment.succeeded", reference, amount, currency, paid_at}.
Idempotency	Store event_id in a processed_events table (UNIQUE). Seen before □ 200 and do nothing.
Work	Look up what the reference belongs to (booking, order, contract...), confirm it inside one transaction, record the event, return 200 within one second. Emails and receipts go to a background task.
Tests	valid □ confirms once; duplicate □ no change; bad signature □ 401; unknown reference □ 200 but logged as an orphan.

Server-Sent Events (SSE)

A normal request is question → answer → goodbye. With **SSE** the browser opens a request and your server keeps it open, pushing small messages down it whenever something. It is the simplest form of “live”: one-way, plain HTTP, no special server. FastAPI's tutorial has a chapter on it. Every project has exactly one stream endpoint (named in its brief). Your stream:

Endpoint	GET /api/v1/stream — returns text/event-stream; auth as the brief says.
Events	Each message is one JSON object: {type, id, data, at}. Send a heartbeat comment every 15 seconds so proxies don't close the connection.
Publishing	When a service changes state, it publishes the event once (an in-process broadcaster is fine; Redis pub/sub is a bonus). Streams never query the database in a loop.
Tests	A test opens the stream, performs the change through the normal API, and asserts the event arrives within one second.

What the team must ship, and when

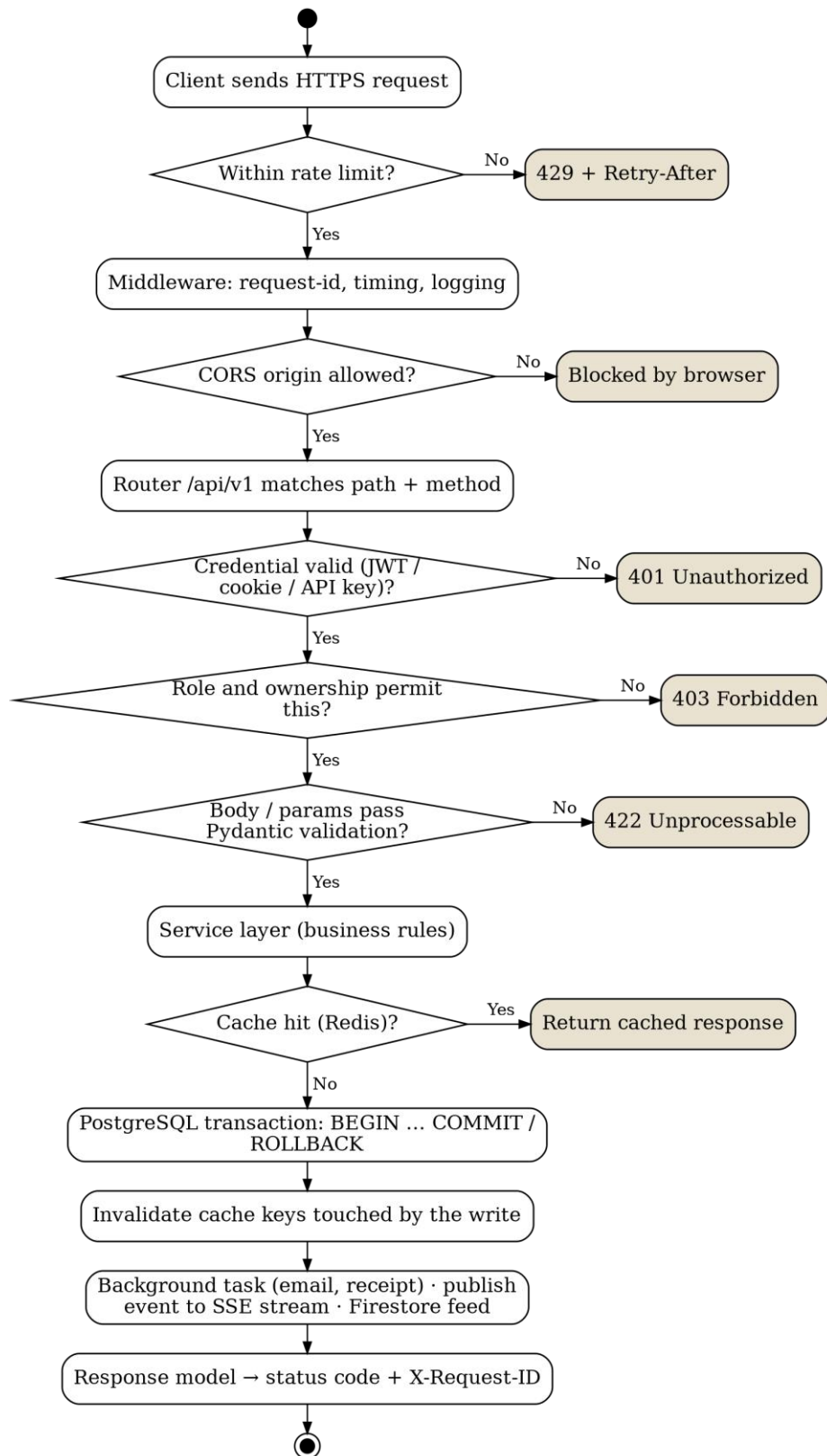
Structure	Package with routers, schemas (in/out), services, repositories, core (config, security, deps), db. Route functions stay thin.
Database	PostgreSQL via SQLModel, the tables in your brief, real constraints and indexes, Alembic migrations, a one-command seed script.
Firestore	Only for the feed / board data your brief names, with a README paragraph on why that data lives there.
Auth	JWT for people, bcrypt-hashed passwords, at least three roles, ownership rules, correct 401 vs 403. Signature for the webhook; API key for any scheduled trigger.
API quality	/api/v1, paginated lists (limit/offset), deliberate status codes, one error shape, filled-in examples and tags.
Performance	One hot read cached in Redis with invalidation on write. Rate limiting on sign-in and public endpoints (429 + Retry-After). Request-id and timing middleware.
Async work	Background tasks for emails/receipts and one scheduled job from your brief (expiring holds, daily sweep, releasing unpaid orders).
Webhook + SSE	As specified on the previous page.
Tests	ε 25 pytest tests on a test database: the acceptance criteria, the five hard-problem tests, the webhook tests, the stream test.
Shipping	Dockerfile, docker-compose (api, postgres, redis), GitHub Actions running tests on every push, deployed on Render/Railway with managed Postgres, live /docs URL.
README + LOG	Setup, demo accounts, ERD, both flowcharts, “The hard problem” and “Why this design” in your own words; LOG.md with a daily entry.

The two weeks, day by day

Day 1	Read the brief twice. Draw the ERD and endpoint list on paper. Write the hard problem in your own words.
Day 2	Design review with instructor (signature). Repo, Docker compose, config, first migration, /docs running, CI green on an empty test.
Day 3	Auth: register, login, JWT, roles, get_current_user, require_role. Tests for 401/403.
Day 4	Core tables and the first two epics' endpoints with tests.
Day 5	Remaining epics. First viva (10 min each). Checkpoint: walking skeleton, at least 10 tests.
Day 6	Webhook endpoint with signature + idempotency, tested with the mock provider.
Day 7	Swap-and-extend morning. Afternoon: SSE stream and its test.
Day 8	The hard problem: write the five tests first (commit them failing), then make them pass.
Day 9	Scheduled job, Redis cache with invalidation, rate limiting, middleware. Checkpoint: hard problem proven.
Day 10	Firestore feed, seed script, error-shape audit, /docs polish.
Day 11	Deploy. Fix everything that only breaks in production. Load the demo data.
Day 12	README, ERD, flowcharts, “The hard problem” and “Why this design” sections. Code review pass against the standards page.
Day 13	Second viva. Freeze the code. Rehearse the demo twice, including a practice live change.
Day 14	Demo day: 15 minutes per team — live demo on /docs, the hard problem, the war story, the live change.

How a request flows

Read this chart once and you understand every FastAPI service you will ever build. Your own brief's flowchart is this chart with your business rules in the middle.



MediStock — Pharmacy Inventory

Team Lion Moons — Gabriel & Loveth

Dispense medicines exactly once, never from an expired batch, even when the network retries.

In one minute

Software for a pharmacy counter. Doctors write prescriptions, pharmacists dispense them, the manager watches stock and expiry dates. The one thing that must never happen: the same dispense being counted twice because the internet was slow and someone clicked again — or medicine leaving the shelf after its expiry date.

Words you need

Batch	A delivery of one drug with one expiry date. Stock is counted per batch, not per drug.
FEFO	First-Expired, First-Out: always dispense from the batch that expires soonest.
Idempotency-Key	A unique id the app sends with a request. If the same key comes twice, you return the first answer and do nothing new.
Basic auth	Username and password sent on every request (used here by the supplier's script, which is a program, not a person).
SSE	A kept-open connection your server uses to push stock alerts to the manager's screen.

A day in the life

Dr. Okafor prescribes 20 Amoxicillin capsules. Pharmacist Kemi dispenses; her app sends Idempotency-Key “k-771”. Your API locks the Amoxicillin batches, skips the one that expired last week, takes 20 from the batch expiring in March, saves the response under “k-771”, commits. Her network drops before she sees the result, and the app retries with the same key — your API returns the saved answer and stock does not move again. That evening, the supplier's script posts a delivery using Basic auth, and the manager's screen shows the new batch appear through the stock stream.

The problem

Community pharmacies lose money in two ways: drugs expire on the shelf, and stock “disappears” because a dispense was recorded twice when the network hiccupped and the pharmacist pressed the button again. MediStock tracks every drug by **batch** with its expiry date, records prescriptions, and dispenses against stock safely. The dispense endpoint must be **retry-safe**: the same request sent twice must reduce stock only once. A daily report tells the manager which batches expire soon.

Who uses it

Pharmacist	Dispenses prescriptions, adjusts stock with a reason.
Doctor	Creates prescriptions with the drugs and quantities.
Store manager	Adds products and batches, sees low-stock, expiry and valuation reports.
Supplier system (machine)	A nightly script from the supplier posts deliveries. It signs in with Basic auth (username and password on each request) — a program, not a person.
Supplier delivery webhook (machine)	The supplier's system calls POST /webhooks/deliveries when a shipment is dispatched. Signed; can arrive twice.

User stories — build them in this order

Accounts & access

- As a pharmacist, I want to sign in so that every dispense is recorded under my name.
- As a manager, I want to create staff logins so that only my team can touch stock.
- As the supplier system, I want a machine login so that deliveries can be posted at night automatically.

Inventory

- As a manager, I want to add a product with batches and expiry dates so that stock is tracked precisely.
- As a manager, I want low-stock and near-expiry reports so that I reorder and discount in time.
- As a pharmacist, I want to adjust stock with a reason so that every correction can be traced.

Dispensing

- As a doctor, I want to create a prescription so that the pharmacist gives exactly what I ordered.
- As a pharmacist, I want to dispense a prescription so that stock is taken from the batch that expires first.
- As a pharmacist, I want to safely retry a dispense that timed out so that stock is never taken twice.
- As a pharmacist, I want expired batches skipped automatically so that I never hand out expired medicine.

Records & notifications

- As a manager, I want a log of every stock movement so that I can explain any number on the shelf.
- As a patient, I want a dispense receipt by email so that I have a record.

Payments & live updates

- As a manager, I want incoming deliveries to appear automatically from the supplier's webhook so that stock is right before the boxes arrive.
- As a manager, I want a live stock-alert stream (SSE) so that low-stock and near-expiry warnings appear as they happen.

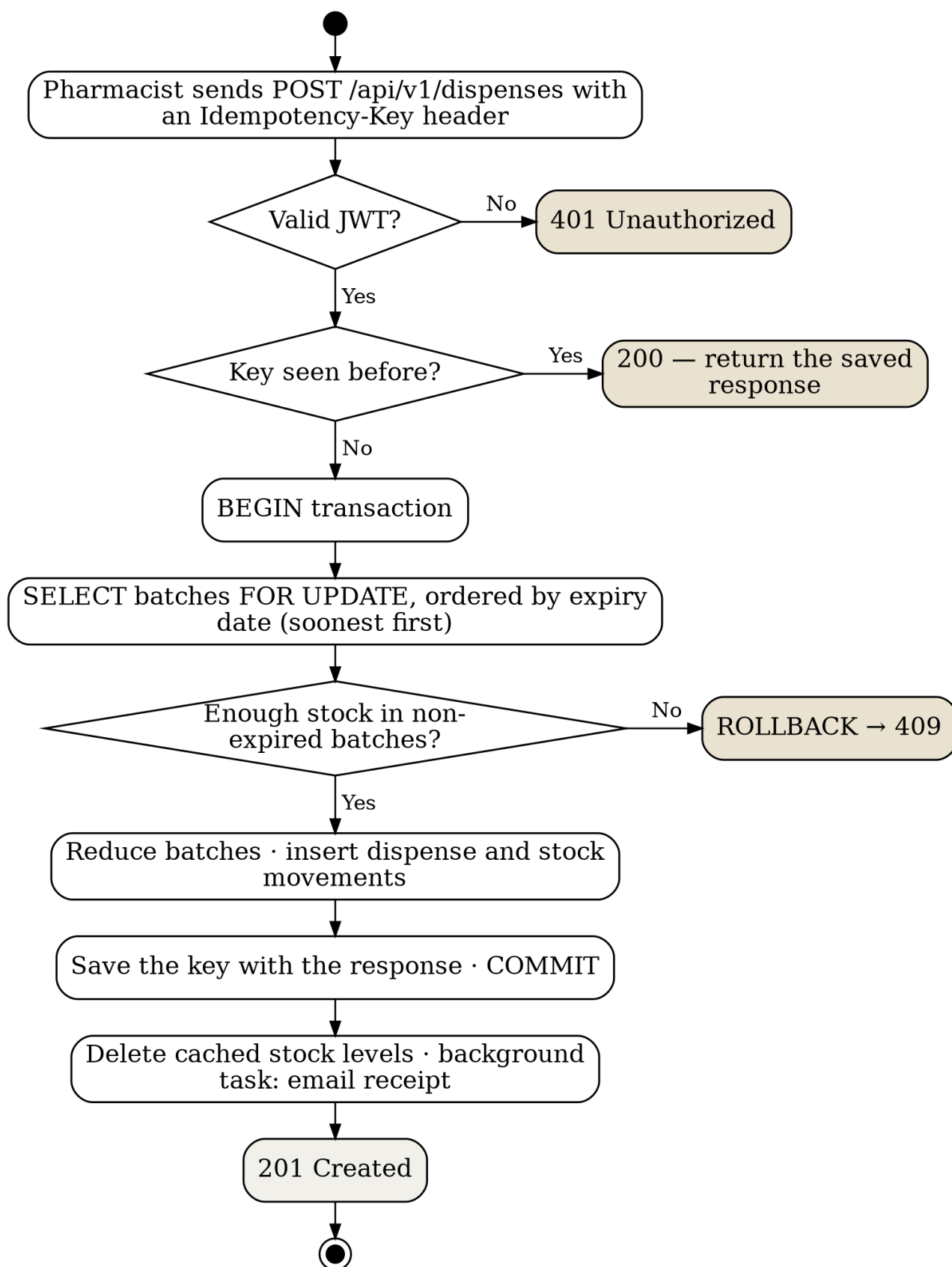
Acceptance criteria — the examiner's test list

Turn every line into one pytest test. The line in bold is the hard problem.

Area	Given / When / Then
Access	• Given no token □ 401. Given a pharmacist token on a manager route □ 403. • Given wrong Basic-auth credentials, when the supplier posts a delivery, then 401.
Dispensing	• Given enough stock, when I dispense with a new Idempotency-Key header, then 201 and stock goes down once. • Given the same Idempotency-Key is sent again, then 200 with the same response as before and stock does not change. • Given the same key but a different body, then 422. • Given not enough non-expired stock, then 409 and nothing changes. • Given an expired batch, when dispensing, then that batch is skipped.
Inventory	• Given a valid product, when created, then 201. Given a negative quantity, then 422. Given an unknown batch, then 404.
Webhook & stream	• Given a signed delivery webhook, then 201 and the batches are created; given the same delivery id again, then 200 and no duplicate batches; given a bad signature, then 401. • Given stock crosses the low-stock threshold, then the alert stream receives an event within one second.

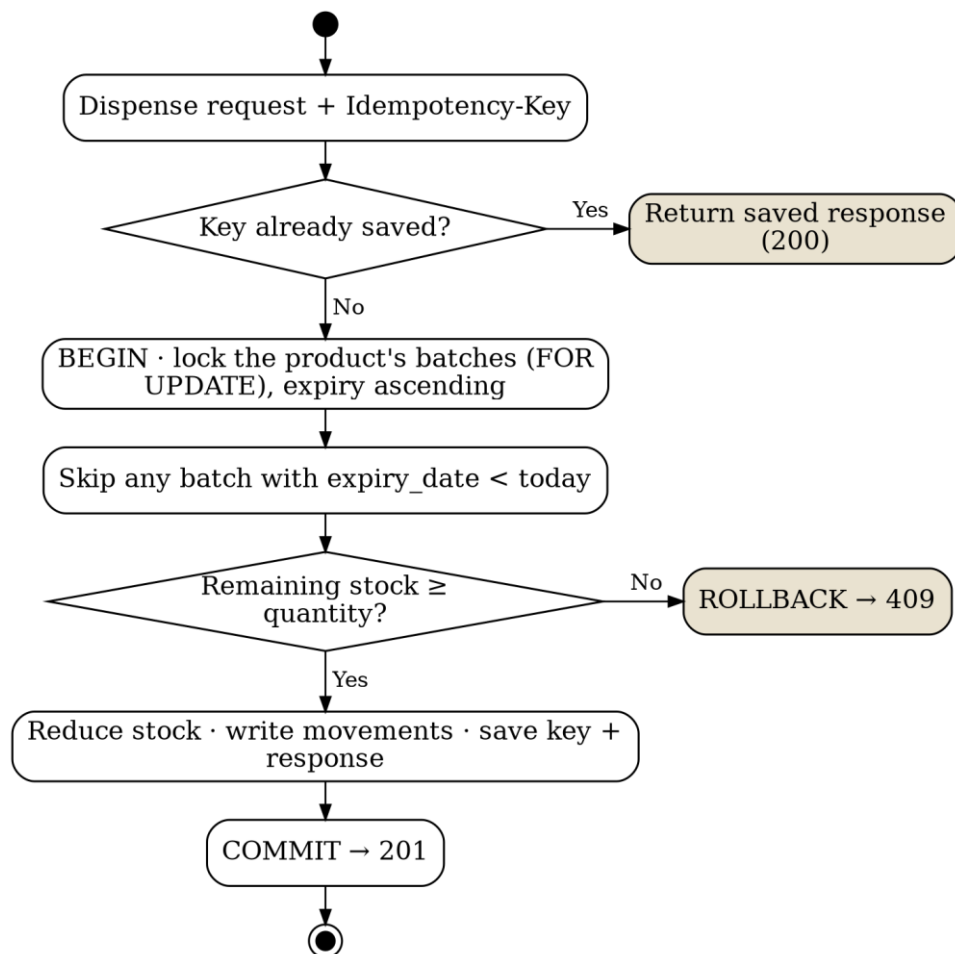
How the key request flows

The most important write in your product — the request the hard problem lives inside. Copy its shape into your service function.



The hard problem — Exactly once, and first-to-expire first

Every dispense request carries an **Idempotency-Key** header — a unique id the client makes up. The first time you see a key you do the work and save the response next to it. Any repeat with the same key gets the saved response and touches nothing. Inside the transaction, take stock from the batch that expires soonest and never from an expired one. The row lock stops two pharmacists from both taking the last unit.



The five tests you must write first (commit them failing, then make them pass)

- Sending the same Idempotency-Key twice reduces stock once and returns the same body both times.
- Two dispenses of the last unit at the same time → one 201, one 409.
- An expired batch is never used; if only expired stock remains → 409.
- Same key with a different body → 422.
- For every batch, the sum of its stock_movements equals its qty_on_hand.

Data model

PostgreSQL (arrows mean foreign keys). Add `processed_events` (`event_id` UNIQUE, `reference`, `processed_at`) for the webhook.

- `users` (`email` UNIQUE, `password_hash`, `role`)
- `products` (`name`, `strength`) — UNIQUE(`name`, `strength`)
- `batches` (`product` → `products`, `batch_no`, `expiry_date`, `qty_on_hand`, `cost`) — UNIQUE(`product`, `batch_no`); index (`expiry_date`)
- `prescriptions` (`patient_name`, `doctor` → `users`, `status`)
- `prescription_lines` (`prescription` → `prescriptions`, `product` → `products`, `qty`)
- `dispenses` (`prescription`, `pharmacist` → `users`, `created_at`)
- `stock_movements` (`batch` → `batches`, `delta`, `reason`, `ref`, `at`)
- `idempotency_keys` (`key` PK, `endpoint`, `body_hash`, `response_json`, `created_at`)

Firestore: stock_events (every movement as a timeline the manager's dashboard reads) and expiry_alerts (batches within 30 days of expiry, refreshed by the daily job).

API surface — your /docs page should match this table

Method	Path (under /api/v1)	Who	Success	Errors
POST	/auth/register	manager	201	403, 409, 422
POST	/auth/login	public	200	401
POST	/products	manager	201	403, 409, 422
POST	/products/{id}/batches	manager	201	403, 404, 422
GET	/inventory?limit&offset;&low;_stock	staff	200	401
POST	/prescriptions	doctor	201	403, 422
GET	/prescriptions/{id}	staff	200	404
POST	/dispenses (Idempotency-Key header)	pharmacist	201 / 200	403, 409, 422
POST	/stock/adjust	pharmacist	201	403, 404, 422
GET	/stock/movements?batch&limit;&offset;	manager	200	403
GET	/reports/low-stock	manager	200	403
GET	/reports/near-expiry?days=30	manager	200	403
GET	/reports/valuation	manager	200	403
POST	/supplier/deliveries (Basic auth)	machine	201	401, 422
POST	/webhooks/deliveries	machine (signed)	201 / 200	401, 422
GET	/stock/alerts/stream	manager	200 (SSE)	401, 403

Backend topics this project proves — expect a viva question on each

Idempotency keys · transactions and row locks · Basic auth for a program vs JWT for people · index on expiry_date · cache with invalidation · daily scheduled job · audit through a movements table. · signed, idempotent webhook · SSE stream (stock/alerts/stream — low-stock and near-expiry alerts).